

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 1 – Git & Docker Technical Assignment

Course Code:	CSA10 / CSA1014	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 1 – Git & Docker Technical Assignment
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	Abdul Riyaz Ahammad	Reg. No.:	192525106
Date:		Duration:	60 minutes

Code of Conduct

I Abdul Riyaz Ahammad(192525106) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Abdul Riyaz Ahammad

Annexure A – Git & Docker Technical Assignment

Instructions to Students:

Each student has been assigned an individual problem statement. Based on the assigned problem statement, prepare a **Git & Docker Technical Assignment** by applying version control and containerization concepts. Your report should include appropriate diagrams, screenshots, commands, and explanations wherever applicable.

Assignment Questions

- 1. Problem Analysis**
 - Explain the assigned problem statement and identify the project objectives, functional requirements, and expected outcomes.
- 2. Project Structure Design**
 - Design and explain the folder structure of your project repository.
 - Justify the purpose of each major folder and file included in the repository.
- 3. Git Repository Initialization**
 - Create a Git repository for the project.
 - Explain the steps involved in repository initialization and describe the purpose of the essential Git files.
- 4. Git Branching Strategy**
 - Design a suitable branching strategy (e.g., Main, Development, Feature, Release, Hotfix).
 - Justify why the selected branching model is appropriate for the assigned project.
- 5. Version Control Workflow**
 - Demonstrate the Git workflow by performing meaningful commits, branch creation, merging, conflict resolution (if applicable), and repository synchronization.
 - Explain the purpose of each Git operation performed.
- 6. Commit History Analysis**
 - Present the commit history of the project.

- Explain how commit messages follow good version control practices and contribute to project maintenance.
- 7. **Docker Environment Design**
 - Explain why Docker is suitable for the assigned project.
 - Identify the application components that require containerization.
- 8. **Dockerfile Development**
 - Design and explain the Dockerfile required to containerize the application.
 - Describe the purpose of each instruction used in the Dockerfile.
- 9. **Docker Compose Design**
 - Develop a Docker Compose configuration for the project.
 - Explain the services, networking, volumes, environment variables, and dependencies used.
- 10. **Container Deployment and Testing**
 - Demonstrate the execution of the application using Docker containers.
 - Explain the testing procedure and provide evidence that the application runs successfully.
- 11. **Benefits of Git and Docker**
 - Discuss how Git and Docker improve collaboration, version management, portability, deployment, scalability, and maintainability for your assigned project.
- 12. **Challenges and Improvements**
 - Identify the challenges encountered during implementation.
 - Suggest possible improvements and future enhancements for the Git workflow and Docker deployment.

Expected Deliverables

- Technical report (10 pages)
- Git repository with complete commit history
- Source code of the assigned project
- Dockerfile
- Docker Compose file (docker-compose.yml)
- Screenshots of Git operations and Docker execution
- Architecture and workflow diagrams

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Project Design(20%)	Clearly explains the problem statement, objectives, requirements, expected outcomes, and well-organized project structure with proper justification.	Good analysis and project structure with minor omissions.	Basic analysis and repository structure with limited justification.	Incomplete or unclear analysis and project organization.
Git Repository & Branching Strategy(20%)	Repository initialized correctly with appropriate branching strategy, clear workflow, and proper repository organization.	Repository and branching strategy are mostly correct.	Basic Git setup with limited branching implementation.	Incorrect or incomplete Git repository and branching strategy.

Version Control Workflow & Commit History(10%)	Demonstrates meaningful commits, branching, merging, synchronization, conflict resolution, and professional commit messages with clear history.	Most Git operations demonstrated correctly with good commit history.	Limited Git workflow and commit practices.	Poor workflow and inadequate commit history.
Docker Implementation(20%)	Well-designed Docker environment, Dockerfile, and Docker Compose configuration with clear explanation of services, networking, volumes, and dependencies.	Functional Docker implementation with minor issues.	Basic Docker implementation with limited explanation.	Incomplete or incorrect Docker implementation.
Deployment, Testing & Results(15%)	Application successfully deployed and tested with appropriate screenshots and evidence of successful execution.	Deployment completed with minor issues.	Partial deployment and testing demonstrated.	Deployment unsuccessful or insufficient evidence.
Documentation, Challenges & Future Enhancements(15%)	Professional report (10–15 pages) with diagrams, screenshots, references, discussion of benefits, challenges, and future improvements.	Good documentation with minor omissions.	Basic documentation with limited discussion.	Poor documentation and insufficient analysis.

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25

1. Problem Analysis

1.1 Industry Context

Warehouses handling thousands of products often face inventory inaccuracies, delayed stock replenishment, misplaced items and inefficient order processing. Traditional inventory audits consume considerable time and labor. These limitations make it difficult for warehouse personnel to maintain accurate stock information and respond quickly to changing inventory conditions.

1.2 Problem Statement

The proposed system is an Intelligent Warehouse Inventory Automation Platform that integrates RFID, barcode scanners, autonomous robots and IoT devices. The platform is designed to monitor inventory, predict stock shortages, optimize storage, automate replenishment and generate inventory reports. The Git and Docker implementation provides controlled software versioning and a reproducible containerized environment for the platform.

1.3 Objectives

- Track inventory accurately using RFID, barcode scanners and IoT-enabled monitoring.
- Predict stock shortages before inventory reaches critical levels.
- Optimize storage by improving item placement and warehouse space utilization.
- Automate replenishment activities when stock reaches defined thresholds.
- Generate warehouse analytics and inventory reports for operational decision-making.

1.4 Expected Outcomes

- Reduced inventory errors.
- Faster order fulfillment.
- Better warehouse utilization.
- Lower operational costs.

1.5 Functional Requirements

ID	Requirement	Description
FR1	Inventory Tracking	Capture and update product quantity and location using RFID/barcode/IoT inputs.
FR2	Stock Monitoring	Monitor stock levels and identify items approaching reorder thresholds.
FR3	Shortage Prediction	Analyze inventory information and identify possible future stock shortages.
FR4	Storage Optimization	Support efficient item placement and warehouse space utilization.
FR5	Automated Replenishment	Generate replenishment actions when stock reaches configured conditions.
FR6	Robot Integration	Provide task information that can be used by autonomous warehouse robots.
FR7	Analytics & Reports	Generate inventory, shortage, replenishment and warehouse performance reports.
FR8	Device Integration	Receive data from RFID readers, barcode scanners and IoT devices.
FR9	Data Management	Persist product, stock, location and transaction information.
FR10	User Access	Provide controlled access for authorized warehouse personnel.

2. Project Structure Design

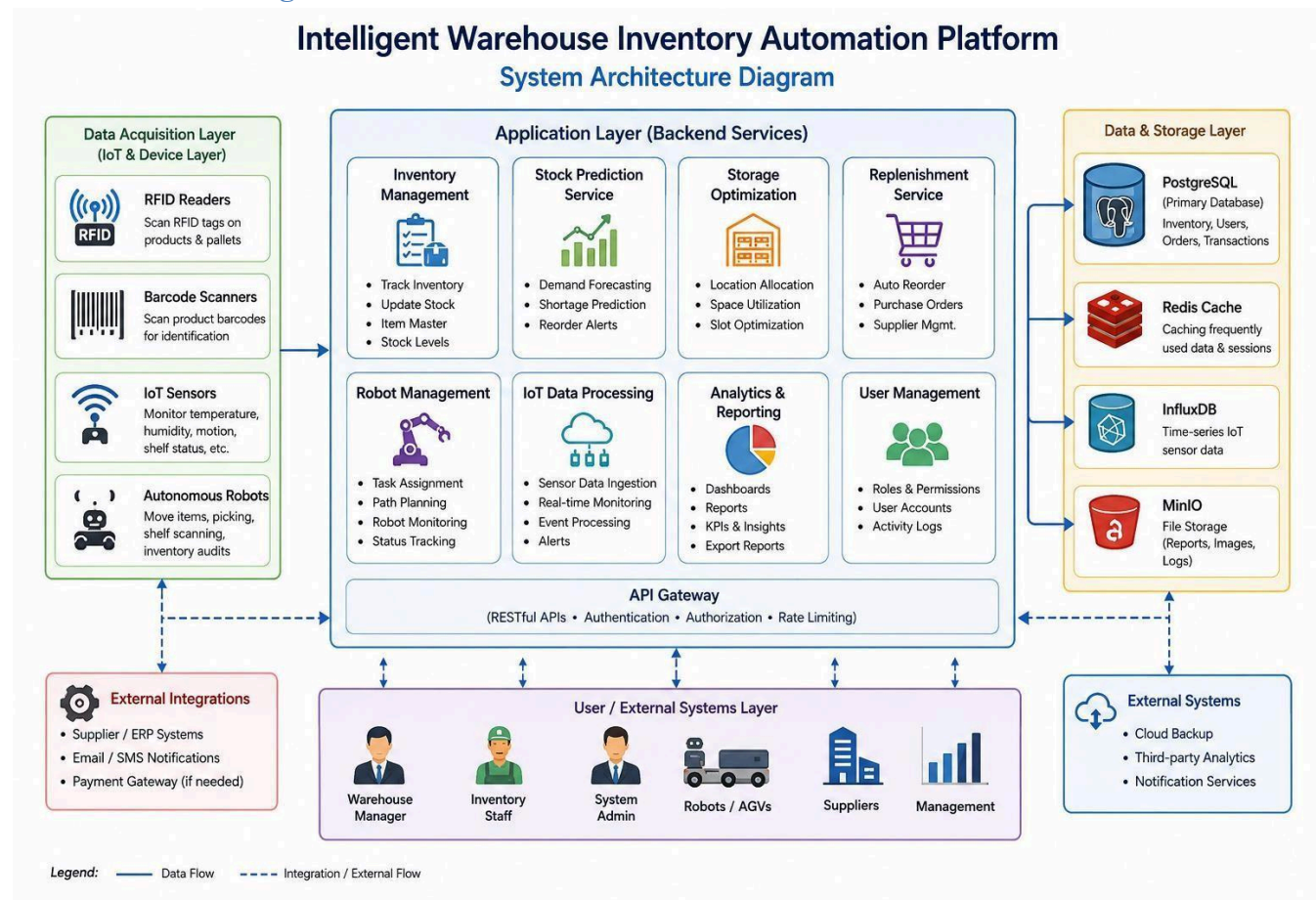
The repository is organized into modules so that the user interface, backend services, device integrations, data management, analytics and deployment configuration remain separated. This structure supports maintainability and allows different features to be developed through independent Git feature branches.

Path	Purpose
warehouse-inventory-platform/	Root repository
frontend/	Dashboard, inventory screens, reports and user interface
backend/	Business logic and REST/API services
backend/routes/	API endpoints for inventory, devices, replenishment and reports
backend/services/	Inventory tracking, shortage prediction, storage optimization and replenishment logic
backend/models/	Product, inventory, warehouse location and transaction models
device-integration/	RFID, barcode and IoT integration adapters/simulation
robot-integration/	Interfaces/tasks for autonomous robot operations
analytics/	Warehouse analytics and report generation
database/	Schema, seed data and database initialization/migration scripts
tests/	Unit and integration tests
docs/	Requirements, architecture and workflow diagrams
Dockerfile	Container image build instructions
docker-compose.yml	Multi-container environment definition
.gitignore	Files excluded from Git
.env.example	Template for runtime configuration without exposing secrets
README.md	Installation, Git and Docker usage instructions

2.1 Architectural Components

Component	Role
RFID / Barcode Layer	Captures product identification and movement events.
IoT Layer	Provides sensor/device information for warehouse monitoring.
Robot Layer	Supports autonomous movement and replenishment-related tasks.
Backend/API Layer	Processes device data, business rules, prediction and replenishment logic.
Database Layer	Stores inventory, product, location, device and transaction information.
Analytics Layer	Produces stock, shortage, replenishment and warehouse utilization reports.
Dashboard Layer	Provides warehouse personnel with monitoring and reporting interfaces.

2.2 Architecture Diagram



3.

Git Repository Initialization

Git is used to manage the source code and configuration of the warehouse inventory platform. It provides a complete history of changes, supports parallel development and enables the team to safely integrate features.

3.1 Initialization Procedure

```
mkdir warehouse-inventory-platform
cd warehouse-inventory-platform
git init
git branch -M main
git status
git add .
git commit -m "chore: initialize warehouse inventory platform"
git remote add origin <repository-url>
git push -u origin main
```

3.2 Essential Git Files and Concepts

Item	Purpose
.git/	Git metadata, objects, references and repository history.
.gitignore	Prevents dependencies, generated files, local configuration and secrets from being tracked.
README.md	Documents project purpose, installation, execution and contribution procedures.
.env.example	Documents required environment variables without storing real secret values.
Remote repository	Provides a shared location for collaboration and backup of source history.

4. Git Branching Strategy

A feature-based branching model with main, develop, feature, release and hotfix branches is suitable for the warehouse platform. Because the system contains several independent modules, each major module can be implemented and tested without directly modifying the stable main branch.

Branch	Purpose	Example
main	Stable, tested versions	main
develop	Integration branch for completed features	develop
feature/*	Individual warehouse feature	feature/inventory-tracking
feature/*	Device/robot integration	feature/rfid-integration
release/*	Release stabilization	release/v1.0
hotfix/*	Urgent correction	hotfix/stock-calculation

The strategy reduces risk by isolating changes. For example, inventory tracking, shortage prediction and automated replenishment can be developed in separate feature branches and integrated into develop after testing.

5. Version Control Workflow

1. Initialize or clone the repository.
2. Create the develop branch from the stable main branch.
3. Create a feature branch for one warehouse function.
4. Implement the function and perform local tests.
5. Use git status to inspect changes.
6. Use git add to stage only relevant files.
7. Create a meaningful commit.
8. Push the feature branch to the remote repository.
9. Merge the completed feature into develop after verification.
10. Resolve conflicts if two branches modify the same code.
11. Run integration tests after merging.
12. Merge the tested release into main and push the stable version.

5.1 Representative Git Commands

```
git checkout -b develop
git checkout -b feature/inventory-tracking
git status
git add frontend/ backend/ database/
git commit -m "feat: implement inventory tracking"
git push -u origin feature/inventory-tracking
git checkout develop
git merge feature/inventory-tracking
git pull origin develop
git push origin develop
git log --oneline --graph --decorate --all
```


5.2 Merge Conflict Resolution

A conflict can occur when two branches modify the same lines or files in incompatible ways. The developer should inspect the conflict, select or combine the correct implementation, remove conflict markers, run tests, stage the resolved files and complete the merge.

git status → edit conflicted file → git add <file> → git commit

6. Commit History Analysis

Commit messages should clearly describe the change. A consistent format such as Conventional Commits makes the project history easier to search and understand.

Commit Message	Purpose
chore: initialize warehouse inventory platform	Initial repository structure
feat: add product and inventory management	Core inventory module
feat: implement RFID and barcode tracking	Device-based identification
feat: add shortage prediction	Stock shortage analysis
feat: add storage optimization	Warehouse placement/space logic
feat: automate replenishment workflow	Replenishment process
feat: add warehouse analytics reports	Reporting module
test: add inventory service tests	Verification and regression support
docker: containerize warehouse platform	Containerization
docs: add deployment and setup guide	Improved maintainability

Evidence required: attach an actual screenshot of the Git repository, branches and git log. The commit history should contain meaningful changes corresponding to the implemented modules rather than artificial or empty commits.

7. Docker Environment Design

Docker is suitable for the Intelligent Warehouse Inventory Automation Platform because the project contains multiple cooperating components. Containerization provides a consistent runtime environment and simplifies deployment of the frontend, backend, database and supporting services.

Container	Purpose
Frontend Container	Warehouse dashboard and user interface.
Backend/API Container	Inventory business logic, APIs and integration services.
Database Container	Persistent product, inventory, location and transaction data.
Optional Analytics/Prediction Container	Can isolate computational analytics or prediction workloads.
Optional Device/Robot Simulation Container	Can simulate RFID, barcode, IoT or robot events during development when physical hardware is unavailable.

8. Dockerfile Development

The exact Dockerfile depends on the technology stack selected for implementation. The following example illustrates a Node.js-based service. If the actual backend uses Python, Java or another runtime, the base image and dependency commands must be changed accordingly.

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json
./ RUN npm ci
COPY .
EXPOSE 5000
CMD ["npm", "run", "start"]
```

8.1 Dockerfile Instruction Explanation

FROM: Selects the required runtime environment.

WORKDIR: Sets the working directory inside the image.

COPY: Copies dependency files and source code into the image.

RUN: Installs required project dependencies during image creation.

EXPOSE: Documents the port used by the service.

CMD: Defines the default startup command.

9. Docker Compose Design

Docker Compose is used to define the multi-service warehouse platform. The Compose configuration can connect the frontend, backend and database through an isolated network and persist database data through a named volume.

services:

```
frontend:
  build: ./frontend
  ports:
    - "3000:3000"
  depends_on:
    - backend
```

```
backend:
  build: ./backend
  ports:
    - "5000:5000"
  environment:
    - DATABASE_URL=${DATABASE_URL}
  depends_on:
    - database
```

```
database:
  image: postgres:16
  environment:
    - POSTGRES_DB=warehouse
```

```
- POSTGRES_USER=${DB_USER}
- POSTGRES_PASSWORD=${DB_PASSWORD}
volumes:
- warehouse_data:/var/lib/postgresql/data
```

```
volumes:
warehouse_data:
```

9.1 Docker Compose Configuration Explanation

Services: Separates the application into independently managed components.

Build: Builds the application image using the corresponding Dockerfile.

Ports: Makes required container services accessible from the host.

Depends_on: Defines startup relationships between services.

Environment variables: Supplies database and runtime configuration without hard-coding secrets.

Volumes: Preserves database information when containers are recreated.

Networking: Allows services to communicate using Compose service names.

10. Container Deployment and Testing

The deployment test demonstrates that the warehouse platform can be built and executed as containers. Actual command output and browser/API screenshots should be inserted into the final submission.

docker compose build

docker compose up -d

docker compose ps

docker compose logs

docker compose down

10.1 Functional Testing

Test Case	Expected Result	Evidence
Inventory tracking	RFID/barcode/IoT event updates the inventory record.	Insert actual screenshot
Stock monitoring	Current stock and reorder level are displayed.	Insert actual screenshot
Shortage prediction	System identifies items likely to become low in stock.	Insert actual screenshot
Storage optimization	System provides storage/placement information or recommendation.	Insert actual screenshot
Replenishment	Low-stock condition generates a replenishment action/task.	Insert actual screenshot
Robot task	Robot integration receives or displays a warehouse task.	Insert actual screenshot
Analytics	Inventory and warehouse reports are generated.	Insert actual screenshot
Database persistence	Data remains available after container restart.	Insert actual screenshot
Container status	Required services remain running successfully.	Insert actual screenshot

10.2 Required Screenshots

- Git repository showing the project files.
- Branch list showing main, develop and feature branches.
- Git commit history using git log.
- Feature branch creation and merge.
- Dockerfile and docker-compose.yml.
- docker compose build output.
- docker compose ps showing running containers.

- Warehouse dashboard running in the browser.
- Inventory/device/analytics functionality working.
- Database or persistence test after container restart.

11. Benefits of Git and Docker

Benefit	Application to Warehouse Platform
Collaboration	Different developers can work on inventory, device integration, prediction, analytics and replenishment features using separate branches.
Version Management	Every change is traceable and previous stable versions can be identified.
Portability	Docker reduces differences between development and deployment environments.
Deployment	Docker Compose starts related services using one reproducible configuration.
Maintainability	Modular repository structure and meaningful commits make changes easier to understand.
Scalability	Services can be separated and scaled as warehouse workload grows.
Reliability	Containerized dependencies reduce environment-related deployment failures.
Reproducibility	The same image/configuration can be used repeatedly for testing and deployment.

12. Challenges and Improvements

12.1 Challenges

- Integrating data from RFID readers, barcode scanners and IoT devices with a common backend format.
- Maintaining accurate inventory when multiple devices report changes at the same time.
- Synchronizing autonomous robot tasks with current stock and warehouse location information.
- Handling database connectivity and service dependencies in a multi-container environment.
- Managing environment variables and credentials securely.
- Resolving Git conflicts when multiple modules share common backend or database files.
- Testing prediction and optimization features using realistic warehouse data.

12.2 Future Enhancements

- Real-time warehouse digital twin and live inventory visualization.
- Advanced machine-learning models for demand and shortage prediction.
- Dynamic robot route optimization.
- Automated alerts for abnormal stock movement and device failures.
- CI/CD pipeline that automatically tests and builds Docker images.
- Container health checks, monitoring and centralized logging.
- Cloud deployment with controlled scaling.
- Role-based access control and secure secret management.
- Automated database backup and recovery.

13. System Requirement and Architectural Justification

The architecture directly supports the assigned warehouse problem by connecting physical identification and sensing technologies to software services that manage inventory information and warehouse decisions. RFID and barcode scanners provide product identification, IoT devices provide monitoring data, and autonomous robots can receive operational tasks. The backend processes these inputs and communicates with the database, analytics, prediction and replenishment components. The dashboard presents operational information to warehouse personnel.

Quality Attribute	Architectural Approach
Performance	Separate services can process inventory events, analytics and reporting without forcing all functions into one component.
Reliability	Version-controlled source code and repeatable container builds reduce software deployment inconsistency.
Maintainability	Feature-based Git branches and modular folders isolate changes.
Portability	Docker packages the runtime environment and dependencies.
Scalability	Additional services or replicas can be introduced as transaction volume increases.
Security	Sensitive configuration should be provided through environment/secret management rather than committed to Git.

Conclusion

The Intelligent Warehouse Inventory Automation Platform addresses major warehouse challenges by integrating RFID, barcode scanners, autonomous robots and IoT devices into a unified inventory platform. The proposed software can track inventory, predict stock shortages, optimize storage, automate replenishment and generate warehouse analytics. Git provides controlled version management and collaborative development, while Docker provides a portable and reproducible environment for the multi-component application. The combination of functional requirements, modular architecture, branching strategy, meaningful commit history, Dockerfile, Docker Compose configuration and actual deployment evidence provides a strong technical basis for the CO3 assessment.

Final Submission Checklist

- ☐ Technical report completed and formatted.
- ☐ Git repository created and shared with complete commit history.
- ☐ Source code uploaded to repository.
- ☐ main/develop/feature branching demonstrated.
- ☐ Dockerfile included.
- ☐ docker-compose.yml included.
- ☐ Architecture and workflow diagrams included.
- ☐ Actual Git screenshots included.
- ☐ Actual Docker execution screenshots included.
- ☐ Application functionality tested and results documented.